

Week06 | 课时 4 | Asset Checks / Metadata / Run Evidence：把“跑过”升级成“可验收”

本课导航

运行完成不是验收，证据完整才是验收	1
这节课解决什么问题	1
参考学习时间	1
学完这一讲，你应该能做到什么	1
本课产出	2
先看质量门禁图	2
1. 三层验收对象不要混用	2
2. 五类 Student Core checks 到底防什么	3
3. Evidence schema 的 required / optional	3
4. Optional 依赖不能 fake passed	4
5. Metadata 放在哪里	5
6. 下游放行决策树	6
7. 最小 evidence 示例	6
自检清单	7
课后最小行动	7
延伸阅读	8

运行完成不是验收，证据完整才是验收

这一讲收紧 Week06 的质量门禁：

asset checks 验证资产状态，run evidence 记录运行事实，二者一起决定下游能不能消费。

这节课解决什么问题

Dagster run 显示 success，但 `ticket_fact_partitioned` 的 required field null rate 超过阈值，Week05 KPI mart 缺失，Week04 lakehouse snapshot 也没有生成。这个时候能不能让 Week08 索引消费？答案不是看 run 是否绿色，而是看 checks、metadata 和 run evidence 是否支持 downstream decision。

Week06 之前，很多系统只能回答“任务跑完了吗”。Week06 之后，我们要回答：

这个资产在这个分区上，是否有足够证据让下游消费？

这要求我们把 checks、metadata 和 evidence 分层。Week06 要把“跑完了”变成“有证据允许下游消费”。

参考学习时间

50-60 分钟

学完这一讲，你应该能做到什么

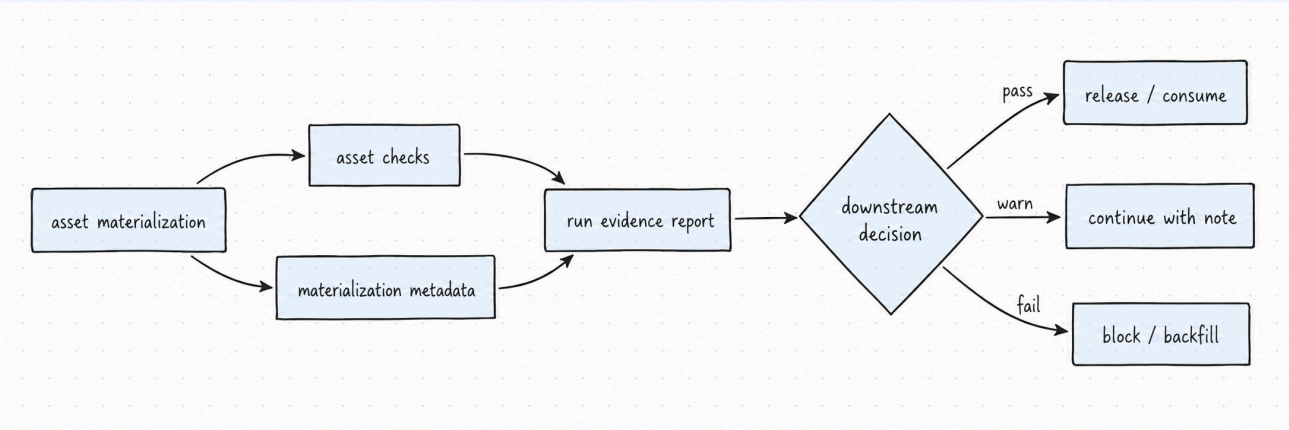
1. 区分 pytest、asset checks 和 run evidence 的职责。
2. 设计 5 类 Student Core asset checks。

- 3. 定义 run evidence schema 的 required / optional 字段。
- 4. 解释 optional Week04 / Week05 依赖为什么不能 fake passed。
- 5. 判断哪些 metadata 放 Dagster, 哪些落 report 文件。

本课产出

- `contracts/run_evidence/week06_run_evidence.schema.json`
- `docs/blueprints/week06/week06-run-evidence-spec.md`
- `reports/week06/run_evidence/`

先看质量门禁图



这张图想让你看懂：materialization finished 只是“动作结束”，不是“资产可消费”。真正让下游放心的是 checks 给出资产状态, metadata 给出关键上下文, run evidence 把运行事实和放行决策落成可复核文件。

1. 三层验收对象不要混用

对象	验什么	例子	谁消费
pytest	代码逻辑和 schema	evidence schema validation、definitions loadable	开发者 / CI
asset check	某个资产当前状态	duplicate、row count、partition completeness	数据工厂 / 下游
run evidence	本次运行事实与决策	status、reason_codes、checks、report_path	讲师 / 下游 / 治理

三者不是互相替代，而是分工互补。pytest 告诉你“代码和契约有没有坏”；asset checks 告诉你“这个资产当前状态能不能信”；run evidence 告诉你“这次运行为什么可以或不可以交给下游”。

项目侧当前对应的测试入口是：

```
# 可执行：已与项目 runbook 对齐
docker compose --profile tools --env-file infra/env/.env.local -f infra/docker-compose.yml run --rm devbox \
    pytest tests/integration/test_week06_asset_checks.py tests/integration/test_week06_run_evidence_generation.py -q
```

怎么看输出：只要其中一类 core check 失败，就不能把 downstream decision 写成 `proceed_to_week07`；如果是 optional 依赖缺失，要写 `skipped/not_available`，不是 `passed`。

2. 五类 Student Core checks 到底防什么

Student Core 不追求搭完整数据质量平台，先把最容易造成“下游误消费”的五类事故挡住。

Check	防什么事故	简单判断逻辑	失败动作	写入 evidence 的字段
<code>manifest_consistency</code>	输入声明缺失或 manifest 里没有资产	至少有 structured manifest, 且 as-sets 非空	<code>hold_downstream</code> , 检查 manifest	<code>checks[]</code> 、 <code>manifest_id</code> 、 <code>reason_codes</code>
<code>row_count_output_correct</code>	产出为空或少数，脚本绿但没数据	<code>input / valid / output row count</code> 不能异常	生成 backfill plan 或人工 review	<code>input_row_count</code> 、 <code>output_row_count</code> 、 <code>checks[]</code>
<code>duplicate_idempotency</code>	重复写入导致 KPI 和索引膨胀	<code>ticket_id</code> 不重复，写入侧有幂等保护	检查 idempotency key, 阻断下游	<code>checks[]</code> 、 <code>metadata.duplicate_c</code> 、 <code>reason_codes</code>
<code>required_field_nullable</code>	必填字段缺失导致 transform / retrieval 失真	<code>ticket_id / status / priority / product_line / created_at</code> 不为空	block / quarantine, 修输入或契约	<code>checks[]</code> 、 <code>reason_codes</code>
<code>partition_completeness</code>	分区缺口，某天数据不完整	selected partition 有 source rows	dry-run backfill	<code>partition_key</code> 、 <code>checks[]</code> 、 <code>metadata</code>

在项目实现里，这五类 check 的纯函数位于 `pipelines/data_factory/checks.py`。它们同时服务测试和 Dagster asset-check wrapper，避免把业务判断复制两遍。

i Asset checks 不是“多写几个断言”

asset check 的目标不是让页面上多几个绿色勾，而是把下游最关心的问题结构化：这个资产、这个分区、这次运行，到底可不可以继续被消费。

3. Evidence schema 的 required / optional

`contracts/run_evidence/week06_run_evidence.schema.json` 是本课的证据契约。它的 `required` 字段不是为了凑 JSON，而是为了让每次运行都能被追踪、复核和交接。

字段	小白解释	工程作用
<code>run_id</code>	这次运行身份证	追踪一次运行
<code>asset_key</code>	哪个资产	绑定 asset graph
<code>partition_key</code>	哪个分区	绑定恢复边界
<code>status</code>	当前结论	<code>success / warning / skipped / not_available / failed</code>
<code>reason_codes</code>	为什么这样判	方便复核和统计

字段	小白解释	工程作用
checks	哪些检查结果	决定能不能放行
report_path	证据在哪里	交接和审计
downstream_decision	下游能不能用	proceed_to_week07 / hold_downstream / manual_review_required / dry_run_only
git_sha	哪版代码	可复现
trace_id	哪条链路	后续 tracing

必填字段建议：

字段	作用
evidence_schema_version	schema 版本
run_id	本次运行标识
asset_key	资产 key
partition_key	分区
status	运行结论
started_at / finished_at	时间边界
report_path	证据文件位置
reason_codes	决策原因

可选字段建议：

字段	为什么可选
manifest_id / batch_id	有些 evidence 来自 observation 或 summary, 不一定绑定具体 batch
source_snapshot_id / output_snapshot_id	Week04 lakehouse 未启用时可能为空
dbt_invocation_id	Week05 analytics 未启用时可能为空
semantic_metric_count	semantic mart 未启用时可能为空
lakehouse_snapshot_id	Week04 lakehouse 状态未启用时可能为空
data_release_id	课程环境可先用 <code>week06-dev-local</code>
trace_id / git_sha	Week11 / Week14 会进一步使用
downstream_decision	根据 checks 和 evidence 决定是否放行
checks	保存结构化 check 结果

4. Optional 依赖不能 fake passed

规则要记住：

缺失的东西不能写 `passed`；未启用的东西不能写 `success`；无法确认的东西不能写可消费。

场景	正确状态	错误写法
Week04 lakehouse 未启用	<code>not_available</code>	<code>passed</code>
Week05 mart 未生成	<code>skipped</code> / <code>not_available</code>	<code>success</code>
check 失败但可继续观察	<code>warning</code> + reason code	不写原因
dry-run 只生成 plan	<code>dry_run_only</code>	<code>proceed_to_week07</code>

如果 Week05 尚未完成，可以写：

```
{
  "status": "not_available",
  "reason_codes": ["week05_analytics_not_available"],
  "dbt_invocation_id": null,
  "semantic_metric_count": null
}
```

同理，Week04 lakehouse 未启用时应写：

```
{
  "status": "not_available",
  "reason_codes": ["week04_lakehouse_not_available"],
  "lakehouse_snapshot_id": null
}
```

⚠️ `skipped` / `not_available` 是诚实状态，`fake passed` 是治理事故

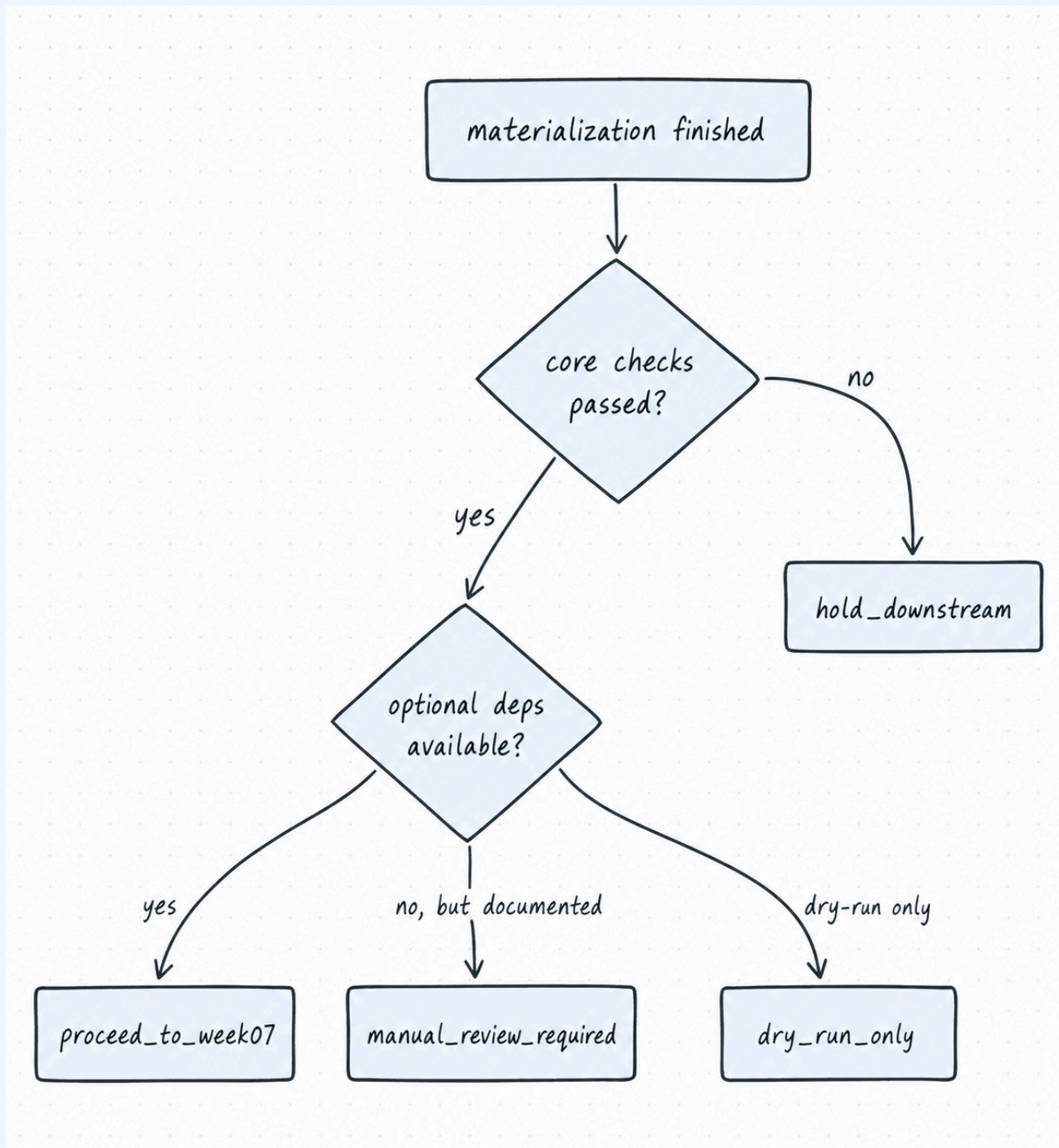
Week06 允许 optional 依赖暂不可用，但必须把不可用原因写进 evidence。后续 Week08 / Week11 / Week14 宁愿看到清楚的 `not_available`，也不能消费一个伪装成 `passed` 的状态。

5. Metadata 放在哪里

信息	放 Dagster metadata	放 report/evidence 文件	原因
row count	是	是	UI 快速看 + 证据归档
partition key	是	是	资产图和 evidence 都需要边界
report path	是	是	从 UI 跳转到证据
大段 JSON	否	是	避免 UI 噪音
full checks result	摘要即可	是	结构化结果要归档
downstream decision	是	是	运维和交接都要看
optional dependency reason	是	是	防 fake passed
git sha / trace id	可摘要	是	后续评测和治理追溯

简单说：Dagster metadata 放“运行时快速定位的信息”，report/evidence 文件放“需要审计、交接、复盘的完整事实”。

6. 下游放行决策树



这棵树的重点是：下游放行不是由 run 的绿色状态决定，而是由 core checks、optional dependency 说明和 dry-run 状态共同决定。课程默认 dry-run 时，即使 core checks 通过，也应该写 `dry_run_only`，不能假装已经可以进入真实消费链路。

7. 最小 evidence 示例

```
{  
  "evidence_schema_version": "week06_run_evidence_v1",  
}
```

```

    "run_id": "week06::2026-04-17",
    "asset_key": "week06/ops/run_evidence_report",
    "partition_key": "2026-04-17",
    "status": "success",
    "started_at": "2026-04-17T10:00:00Z",
    "finished_at": "2026-04-17T10:02:30Z",
    "report_path": "reports/week06/
run_evidence/week06_ops_run_evidence_report_2026-04-17.json",
    "reason_codes": ["dry_run_no_db_write"],
    "input_row_count": 3,
    "output_row_count": 0,
    "data_release_id": "week06-dev-local",
    "downstream_decision": "dry_run_only",
    "checks": [
      {
        "name": "manifest_consistency",
        "status": "passed",
        "reason_codes": [],
        "metadata": {"structured_manifest_count": 1}
      },
      {
        "name": "duplicate_idempotency",
        "status": "passed",
        "reason_codes": [],
        "metadata": {"duplicate_count": 0}
      }
    ]
  }
}

```

这个示例刻意写成 `dry_run_only`：它说明 Week06 课堂默认模式下可以验证流程和证据，但不把结果伪装成真实发布。常见错误是 `evidence_schema_version` 写错、`reason_codes` 不是数组、`checks[]` 缺少 `reason_codes`、`report_path` 指向不存在文件。

! 核心判断

可验收的数据资产必须同时有运行结果、质量判断和可追溯证据。

自检清单

- ☐ 我能说清 `pytest`、`asset checks` 和 `run evidence` 的分工。
- ☐ 我能列出 5 类 Student Core checks，并说清每类防什么事故。
- ☐ 我能区分 `evidence required` 和 `optional` 字段。
- ☐ 我知道 `optional` 依赖未启用时要写 `not_available` / `skipped`。
- ☐ 我能说明哪些 `metadata` 适合放在 UI，哪些应该落 `report`。
- ☐ 我能根据 `checks` 和 `evidence` 写 `downstream decision`。

课后最小行动

补一份 `evidence` 字段表：

```
## Week06 run evidence fields
```

- Required:
- Optional:
- Status values:
- Reason code examples:
- Downstream decision rule:
- Metadata in Dagster UI:
- Metadata in report archive:

延伸阅读

- Dagster: [Asset checks](#)¹
- Dagster: [Asset materializations and metadata](#)²
- Dagster: [Asset observations](#)³
- OpenLineage: [Facets](#)⁴
- 项目证据契约: `contracts/run_evidence/week06_run_evidence.schema.json`

¹Dagster asset checks 用于描述资产状态是否满足条件。课程里只做五类 Student Core checks, 不扩展成完整质量平台。

²Dagster materialization metadata 适合放 row count、partition key、report path 等快速定位信息；大段 JSON 仍应落到 report/evidence 文件。

³Asset observations 用于记录外部资产状态，不代表本周成功生成了这个资产；这就是 optional Week04 / Week05 依赖不能 fake passed 的原因。

⁴OpenLineage facets 是后续 Week14 的治理延伸，本课只用它作为“结构化运行事实未来可以进入 lineage/eval/release 系统”的预告。